

IT004

CƠ SỞ DỮ LIỆU

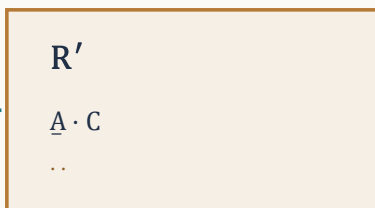
BIÊN SOẠN: VÕ TRỌNG PHÚC



CƠ SỞ DỮ LIỆU

BIÊN SOẠN: VÕ TRỌNG PHÚC

$$\pi_{A,C}(\sigma_{B=1}(R))$$



IT004

CƠ SỞ DỮ LIỆU

BIÊN SOẠN: VÕ TRỌNG PHÚC

R1

PK

A · B · C

R2

PK → FK

B · C

R3

X → Y

...

THÔNG TIN XUẤT BẢN

Một trang về bản thảo

Bản proof này thử nghiệm một ngôn ngữ chế bản cho cẩm nang lý thuyết IT004: trang giấy sáng, lưới A4 ổn định, chữ dễ đọc và hình minh họa giải thích được cơ chế.

PHẠM VI

Nội dung được giữ nguyên tinh thần

Khái niệm, ký hiệu và ví dụ đại diện được chọn từ handbook hiện tại và thuật ngữ UIT đã kiểm tra trong Phase A. Đây là proof nội bộ, chưa phải bản phát hành v1.1.

QUYỀN

Ghi nguồn ở nơi phù hợp

Đề thi, slide, giáo trình và tài liệu Microsoft vẫn thuộc chủ sở hữu tương ứng. Bản proof chỉ dùng diễn giải độc lập, hình tự vẽ và đường dẫn nguồn; không phân phối bản scan.

Ghi nhớ

Đây không phải tài liệu chính thức của UIT.

Bản thảo được biên soạn độc lập cho mục đích học tập; quyết định chuyển đổi toàn bộ handbook chỉ thực hiện sau khi human review của proof này.

Baseline nội dung: Phase A frozen commit 6aef91eb2cb4a0b41827573bc03ec55640d19786.

MỤC LỤC MẪU

Bản đồ đọc

Mỗi chương bắt đầu bằng một câu hỏi, đi qua hình hoặc bảng giải thích, rồi mới đặt công thức và bài tập vào đúng ngữ cảnh.

Chương 1 — Vì sao cần DBMS?	06–07
File system và DBMS · ba mức kiến trúc · độc lập dữ liệu	
Chương 2 — Từ thế giới thực đến quan hệ	08–09
Relation, tuple, attribute · khóa · ER → relational schema	
Chương 3 — Đại số quan hệ	10–11
Selection · projection · join · division và điều kiện “tất cả”	
Chương 5 — Giữ dữ liệu đúng	12
Ràng buộc toàn vẹn · bảng tầm ảnh hưởng	
Chương 6 — PTH và chuẩn hóa	13–14
Bao đóng · khóa ứng viên · 3NF/BCNF · phân rã	
Bài tập và chẩn đoán	15–17
Bài tập · lời giải mẫu · sai lầm thường gặp	

Cách đọc: Hình cho trực giác, bảng cho đối chiếu, công thức cho độ chính xác, bài tập cho kiểm chứng.

CÁCH HIỂU

File system và DBMS

Cùng một dữ liệu có thể được lưu trong các tệp rời hoặc được quản lý bởi một hệ quản trị. Khác biệt nằm ở việc ai chịu trách nhiệm giữ cho dữ liệu dùng được khi nhiều người cùng làm việc.

Bảng 1. Hai cách tổ chức dữ liệu trong cùng một bài toán

Khía cạnh	File system	DBMS
Trùng lặp	Mỗi ứng dụng có thể giữ một bản riêng.	Lược đồ chung giảm dư thừa có kiểm soát.
Toàn vẹn	Quy tắc rải trong nhiều chương trình.	PK, FK, CHECK và transaction bảo vệ trạng thái.
Truy cập đồng thời	Dễ ghi đè hoặc đọc trạng thái dở dang.	Điều phối khóa, transaction và phục hồi.
Thay đổi cấu trúc	Đổi định dạng thường kéo theo sửa ứng dụng.	DBMS cung cấp mức nhìn và data independence.

Ý chính

DBMS không chỉ là nơi chứa bảng; nó quản lý ngữ nghĩa, quyền truy cập, đồng thời và phục hồi.

Nhận xét

Nếu chỉ nhìn vào vài tệp dữ liệu, ta chưa thấy được các quy tắc mà hệ thống phải giữ.

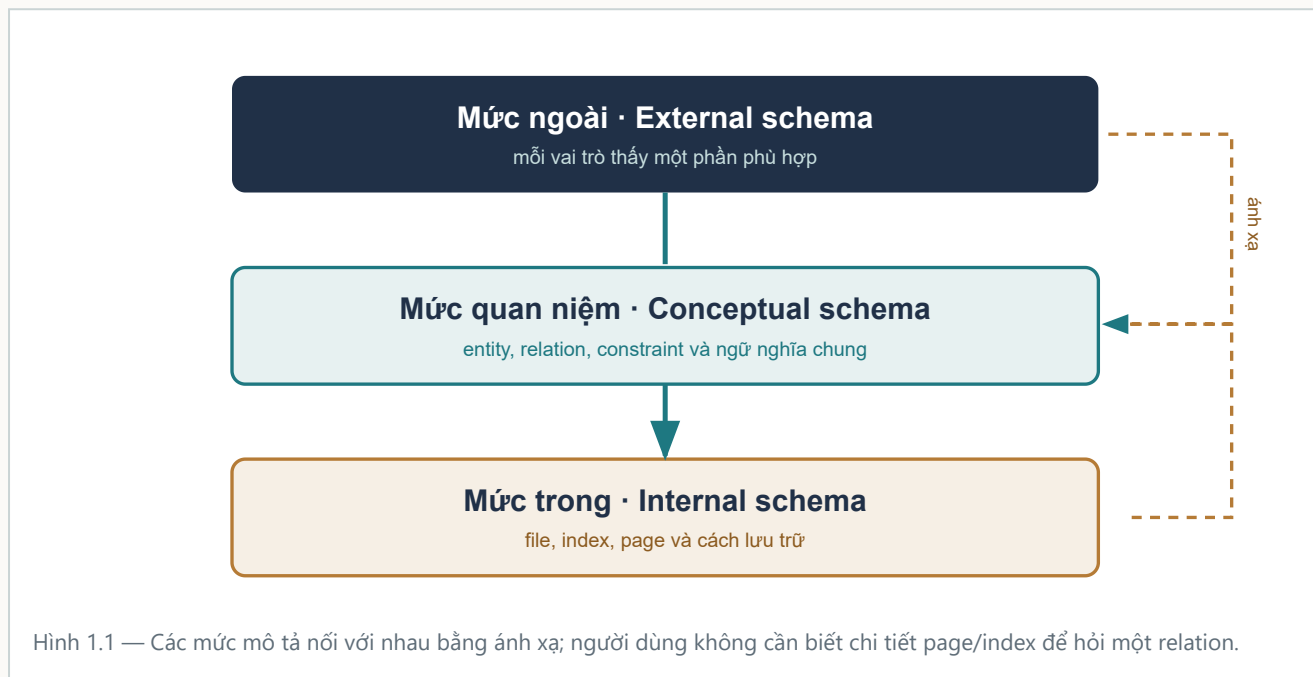
Một phép so sánh ngắn

Danh sách điểm của một lớp có thể là CSV. Nhưng khi cần cập nhật điểm, tra cứu học viên, bảo đảm mỗi mã là duy nhất và phục hồi sau lỗi, ta cần lớp quản trị của DBMS.

SƠ ĐỒ CƠ CHẾ

Ba mức nhìn dữ liệu

Một thay đổi ở nơi lưu trữ không nhất thiết phải làm thay đổi cách người dùng nhìn dữ liệu. Ba mức mô tả giúp tách giao diện, lược đồ logic và cách cất giữ.



- 1 Yêu cầu đổi index:** thay đổi ở mức trong.
- 2 Lược đồ quan niệm giữ nguyên:** relation và constraint không đổi.
- 3 Ứng dụng tiếp tục chạy:** đây là trực giác của độc lập dữ liệu vật lý.

THUẬT NGỮ

Relation, tuple, attribute và khóa

Một relation là một tập các tuple có cùng schema. Đọc đúng từng lớp giúp ta không nhầm “cột” với “giá trị trong một dòng”, và không nhầm khóa với một thuộc tính bất kỳ.

SCHEMA

HOCVIEN

HOCVIEN(MAHV, HO, TEN, MALOP)

Schema nói relation có những thuộc tính nào.

INSTANCE

Một thể hiện

Hai tuple khác nhau có thể cùng schema; instance thay đổi theo thời điểm và dữ liệu.

Bảng 2. Đọc một relation mẫu

MAHV	HO	TEN	MALOP
HV01	Nguyễn	An	L01
HV02	Trần	Bình	L01
HV03	Lê	Chi	L02

Ghi nhớ

Tuple = một dòng; attribute = một cột; domain = miền giá trị của cột; khóa = thuộc tính hoặc tập thuộc tính định danh duy nhất tuple.

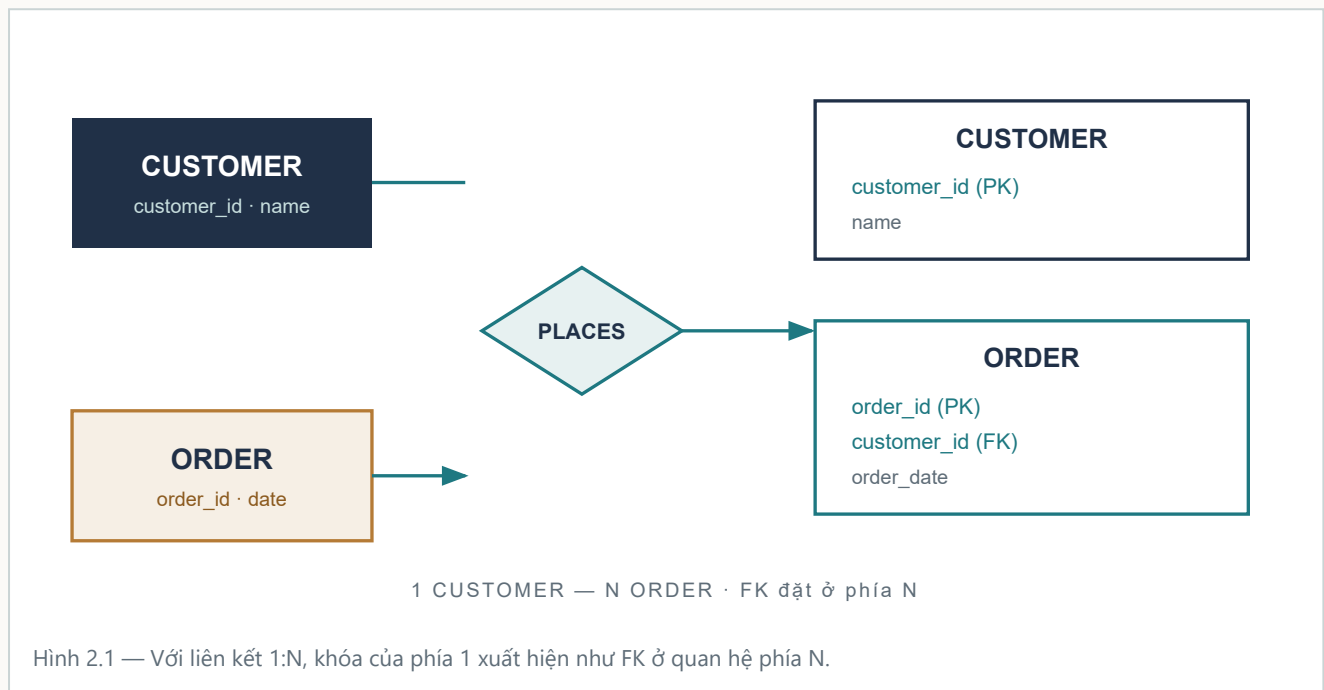
Khóa không chỉ để “đánh số”

Superkey xác định duy nhất tuple. **Candidate key** là superkey tối thiểu. Một relation có thể có nhiều candidate key; ta chọn một làm primary key, các liên kết sang relation khác thường dùng foreign key.

VÍ DỤ

Đặt khóa vào đúng quan hệ

Khi ánh xạ, câu hỏi quan trọng là: một thực thể được nhận diện bằng gì, và mối kết hợp cần giữ những khóa nào để không mất ngữ nghĩa?



- 1 Tạo relation cho mỗi entity và giữ PK của entity đó.
- 2 Đọc lực lượng: CUSTOMER 1 — N ORDER.
- 3 Đặt `customer_id` vào ORDER làm FK; không tạo cột “PLACES” thừa.

THEO DÕI TRẠNG THÁI

Selection, projection và join

Đừng nhảy thẳng tới biểu thức cuối. Theo dõi relation trung gian giúp kiểm tra điều kiện, tên thuộc tính và bội số dòng ở từng bước.

1 **Input:** CUSTOMER(customer_id, name) và ORDER(order_id, customer_id, total).

2 **Selection:** $\sigma_{\text{total} > 1000}(\text{ORDER})$ giữ các tuple thỏa điều kiện.

3 **Join:** nối qua `customer_id` để lấy tên khách.

4 **Projection:** chỉ giữ name, total sau khi đã có đủ thuộc tính cần dùng.

```
 $\pi$  name, total  
(CUSTOMER  $\bowtie$  CUSTOMER.customer_id = ORDER.customer_id  
 $\sigma$  total > 1000 (ORDER))
```

Công thức chỉ là bản tóm tắt; relation trung gian là cách kiểm tra nó.

SELECTION σ

Lọc *tuple*; số cột giữ nguyên.

PROJECTION π

Lọc *attribute*; có thể loại bản trùng theo ngữ nghĩa relation.

Sai lầm thường gặp

Chiếu bỏ `customer_id` trước khi join sẽ làm mất cầu nối giữa hai relation. Đổi tên bằng `ρ` chỉ thay nhãn, không tạo tuple mới.

CÁCH HIỂU

“Tất cả” và phép chia

Phép chia trả về những giá trị ở phía X ghép được với *mọi* giá trị trong divisor. Khi viết SQL, trực giác này thường xuất hiện dưới dạng hai lớp **NOT EXISTS**.



X is kept only when no required Y is missing

Hình 3.1 — Divisor định nghĩa “tất cả”. Đổi divisor là đổi câu hỏi.

```
SELECT DISTINCT e.X
FROM ENROLL AS e
WHERE NOT EXISTS (
  SELECT 1 FROM REQUIRED AS r
  WHERE NOT EXISTS (
    SELECT 1 FROM ENROLL AS e2
    WHERE e2.X = e.X AND e2.Y = r.Y
  )
);
```

Dùng **DISTINCT** vì đại số quan hệ làm việc trên tập, còn SQL giữ ngữ nghĩa bag nếu không yêu cầu loại bản trùng.

Khi làm bài

Hãy nói thành lời: “Không tồn tại một yêu cầu nào mà X chưa đáp ứng”. Câu nói này giúp kiểm tra đúng hai lớp phủ định.

VÍ DỤ + BẢNG TẦM ẢNH HƯỞNG

Một quy tắc, ba thao tác

Phát biểu RBTV trước, xác định quan hệ và thuộc tính chịu ảnh hưởng sau. Bảng tầm ảnh hưởng là bản đồ kiểm tra, không phải danh sách trigger viết theo cảm tính.

RBTV mẫu

Với mọi dòng **HOCVIEN**, nếu có **MALOP** thì giá trị đó phải tồn tại trong **LOP(MALOP)**.

Bảng 5.1 — RBTV tham chiếu HOCVIEN → LOP

Quan hệ / thao tác	Thêm	Xóa	Sửa	Lý do
HOCVIEN	+	–	+(MALOP)	Thêm/sửa FK có thể tạo tham chiếu mờ côi.
LOP	–	+	+(MALOP)	Xóa/sửa PK có thể làm HOCVIEN mất cha.

ĐIỀU KIỆN

$\forall h \in \text{HOCVIEN}: h.\text{MALOP} = \text{NULL} \vee h.\text{MALOP} \in \pi_{\text{MALOP}}(\text{LOP})$

CÁCH KIỂM TRA

Kiểm tra tập giá trị bị tác động sau thao tác; khi có nhiều dòng, dùng logic tập hợp.

Nhận xét

Dấu + chỉ nói “có khả năng vi phạm”. Nó không tự quyết định hành động CASCADE hay NO ACTION.

CHẠY BAO ĐÓNG

Từ X đến X^+

Bao đóng là một quy trình lặp: bắt đầu bằng X , áp dụng PTH có vế trái đã biết, rồi dừng ở điểm cố định. Đây là cách kiểm tra siêu khóa và thành viên của PTH.

VÍ DỤ

$R(A, B, C, D, E), F = \{A \rightarrow BC, B \rightarrow D, CD \rightarrow E\}$

- 0 **Bắt đầu:** $A^+ = \{A\}$.
- 1 **$A \rightarrow BC$:** $A^+ = \{A, B, C\}$.
- 2 **$B \rightarrow D$:** $A^+ = \{A, B, C, D\}$.
- 3 **$CD \rightarrow E$:** $A^+ = \{A, B, C, D, E\} = R$.

Kết luận

Vì $A^+ = R$, A là superkey. Muốn biết A có là candidate key, còn phải kiểm tra tính tối thiểu.

Cách kiểm tra kết quả

Thử bỏ từng thuộc tính khỏi X . Nếu bao đóng không còn là R , thuộc tính đó là cần thiết.

NHẬN XÉT

3NF, BCNF và phân rã

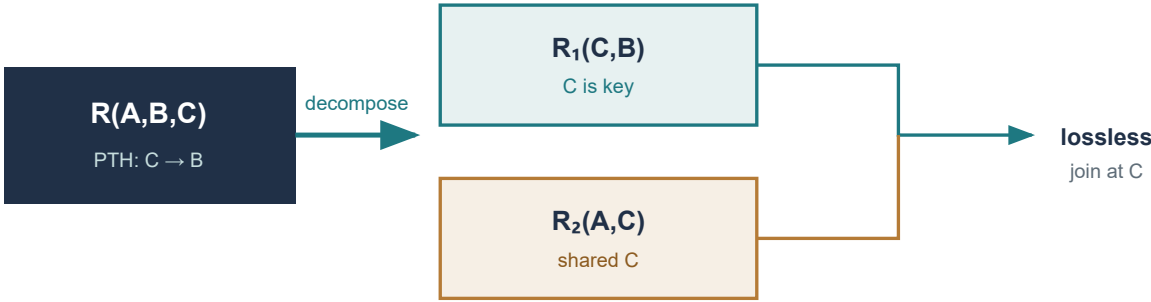
Chuẩn hóa không phải là tách bảng càng nhiều càng tốt. Ta tìm khóa, đọc PTH và chọn phân rã vừa loại dị thường vừa giữ được thông tin cần thiết.

Bảng 6.1 — Tiêu chuẩn chính thức

Dạng chuẩn	Điều kiện cốt lõi	Câu hỏi kiểm tra
2NF	1NF và không có phụ thuộc bộ phận của thuộc tính non-prime vào khóa ghép.	Khóa ứng viên có nhiều thuộc tính không?
3NF	Với $X \rightarrow A$ không hiển nhiên: X là superkey hoặc A là prime.	Về trái chưa là superkey thì A có prime không?
BCNF	Với mọi $X \rightarrow A$ không hiển nhiên, X phải là superkey.	Có PTH nào có determinant không là superkey?

Ghi nhớ

BCNF chặt hơn 3NF: ngoại lệ “vế phải là prime” chỉ cứu được 3NF, không cứu được BCNF.



Hình 6.1 — Phân rã cần được kiểm tra lossless join và dependency preservation, không chỉ nhìn số lượng bảng.

BÀI TẬP

Một bài, bốn điểm kiểm tra

Câu hỏi**12 phút**

Cho hai relation:

ENROLL(MAHV, MAMH) và REQUIRED(MAMH)

Viết biểu thức đại số quan hệ trả về các MAHV đã đăng ký **tất cả** môn trong REQUIRED. Sau đó nêu một cách viết SQL tương đương bằng hai lớp **NOT EXISTS**.

Checklist trước khi nộp

1. Divisor có đúng là tập môn bắt buộc không?
2. Kết quả cuối cùng chỉ còn thuộc tính MAHV ?
3. Hai lớp phủ định trong SQL có diễn đạt “không có môn bắt buộc bị thiếu” không?
4. Nếu REQUIRED rỗng, bạn đã hiểu ngữ nghĩa “tất cả” chưa?

Ghi nhớ

Hãy giải bằng relation trung gian trên giấy nháp trước khi rút gọn thành một dòng công thức.

MỞ RỘNG

Thay REQUIRED bằng một relation có thêm học kỳ. Bạn phải quyết định divisor và điều kiện join có cần thêm thuộc tính hay không.

CÁCH GIẢI

Theo dõi “không bị thiếu”

Đại số quan hệ:

$$\pi_{\text{MAHV}}(\text{ENROLL}) - \pi_{\text{MAHV}}((\pi_{\text{MAHV}}(\text{ENROLL}) \times \text{REQUIRED}) - \text{ENROLL})$$

Phân trong ngoặc tạo mọi cặp học viên–môn bắt buộc, rồi trừ những cặp đã có. Lấy MAHV của các cặp còn thiếu và trừ khỏi toàn bộ học viên.

```
SELECT DISTINCT e.MAHV
FROM ENROLL AS e
WHERE NOT EXISTS (
  SELECT 1
  FROM REQUIRED AS r
  WHERE NOT EXISTS (
    SELECT 1
    FROM ENROLL AS e2
    WHERE e2.MAHV = e.MAHV
      AND e2.MAMH = r.MAMH
  )
);
```

Hai lớp NOT EXISTS: lớp ngoài chọn học viên; lớp trong tìm một môn bắt buộc chưa có.

Cách kiểm tra kết quả

Với MAHV = HVO1 và REQUIRED = {CSDL, HÐH}, nếu HVO1 chỉ có CSDL thì lớp trong tìm thấy HÐH bị thiếu và HVO1 bị loại. **DISTINCT** loại các dòng ENROLL lặp cùng MAHV; phép chiếu trong đại số quan hệ vốn là tập.

ĐÚNG

Divisor được giữ nguyên là REQUIRED.

CẦN TRÁNH

Đếm số môn mà không kiểm tra trùng hoặc môn ngoài divisor.

SAI LẦM THƯỜNG GẶP

Đọc lỗi như đọc dữ liệu

Một đáp án sai thường để lại dấu vết cụ thể. Gọi đúng tên lỗi giúp sửa cấu trúc, không chỉ sửa một ký tự.

Bảng 7.1 — Triệu chứng và nguyên nhân

Triệu chứng	Nguyên nhân có thể	Cách kiểm tra
Kết quả division rỗng	Divisor nhầm relation hoặc lớp NOT EXISTS bị đảo.	Viết câu “không có yêu cầu bị thiếu” bằng tiếng Việt.
SQL trả X/MAHV lặp	SQL dùng bag semantics; một X có nhiều dòng ENROLL.	Dùng DISTINCT hoặc tạo miền X bằng projection trước.
3NF đúng nhưng BCNF sai	Determinant chưa là superkey nhưng vế phải là prime.	Tính closure của vế trái; không dùng ngoại lệ 3NF cho BCNF.
FK bị mờ côi	Thêm/sửa bảng con hoặc xóa/sửa bảng cha không kiểm tra.	Đọc bảng tầm ảnh hưởng theo từng thao tác.

Chẩn đoán

Nếu một công thức không thể giải thích bằng câu nói và relation trung gian, hãy quay lại mô hình trước khi rút gọn.

Ba câu hỏi kết thúc

1. Quan hệ đầu vào và đầu ra có đúng schema không?
2. Điều kiện logic có giữ đúng “tất cả”, “không có”, hoặc “ít nhất” không?
3. Một người khác có thể làm theo các bước để tái tạo kết quả không?